

TERRITORIAL.IO

Autonomous Human-Level Agent

Full Technical Project Report

Chrome Manifest V3 Extension · Single-Player Bot · Source-Faithful Architecture

Version **9.9.2** · Report date: **2026-07-31**

Project path: Desktop/territorial-chrome-extension

Target: <https://territorial.io/>

This document describes the design, reverse-engineering basis, module inventory, attack pipeline, safety systems, performance model, installation, and known limitations of the Territorial.io Chrome extension bot.

Table of Contents

1. Executive Summary
2. Project Goals and Scope
3. System Architecture
4. Repository Structure and Module Inventory
5. Game Reverse Engineering and Source Map
6. Match Lifecycle and Arming Model
7. Sensing: Vision, Grid, Borders, Coordinates
8. Decision: Strategy, Economy, Hard-Mode Profiles
9. Actuation: Internal API vs Canvas Sword Attacks
10. Troop Percentage Control
11. UI Chrome Safety and Username-Popup Prevention
12. Performance Model (Uncapped FPS)
13. User Interface: HUD, Popup, Hotkeys
14. Installation, Operation, and Debugging
15. Version History and Problem Log
16. Limitations, Risks, and Future Work
17. Appendix: Key Symbols and Quick Reference

1. Executive Summary

This project is a **Chrome Manifest V3 extension** that automates single-player play on **Territorial.io**. It combines (a) a MAIN-world hook into the live game runtime, (b) an isolated-world orchestrator with computer vision and strategy modules, and (c) a dual actuation path: preferred **internal engine calls** (bB.hZ.hg / expand / crush) plus a reliable **canvas two-click sword attack** fallback when needed.

Design philosophy is **source-faithful**: Very Hard bot tables and decision order (expand free land first, then crush weakest / pressure closest) were ported from a readable game dump and mapped onto the live minified desktop site. The bot deliberately **does not arm until the human presses Play and clicks a spawn location**, avoiding menu hijacking and accidental Quit/username flows.

Current release **v9.9.2** features: perimeter-only adjacent attacks, adaptive troop commit (~12–45%), uncapped decision loop (every requestAnimationFrame), hard exclusion of bottom-left Quit logo chrome (which previously caused the username window), and troop % forced via storage + slider reload rather than bar drag.

2. Project Goals and Scope

2.1 Primary goals

- Play single-player Territorial.io at near human / Very Hard pace without taking over the OS mouse permanently.
- Respect real game rules: adjacency, troop tax, density soft-caps, multi-front pressure.
- Arm only after Play + map spawn confirmation (pixel territory evidence).
- Prefer internal APIs; keep canvas click path as robust fallback (open green sword → confirm).
- Keep the user's real pointer free; pause bot while user holds mouse down.

2.2 In scope

- Chrome MV3 extension for territorial.io (and local hosts for testing).
- Single-player match automation after human spawn.
- HUD telemetry, popup settings, hotkeys (Z/C/V/B).
- Vision pipeline for borders / free land / enemies when internal neighbor data is incomplete.

2.3 Out of scope / not claimed

- Guaranteed multiplayer fairness or anti-cheat compliance (extension automates a web game).
- Perfect play / rank climbing claims.
- Mobile app packaging (desktop Chrome focus).
- Server-side botting.

3. System Architecture

3.1 Dual JavaScript worlds (MV3)

Chrome content scripts can run in two worlds. This project uses both:

World	Entry	Role
MAIN	content/main-hook.js @ document_start	Runs in the page's JS realm. Patches/exports game objects as window.__TIO_GAME__, implements setTroopRatio, hg/pZ attacks, neighbor checks, state snapshot.

World	Entry	Role
Isolated	content/*.js @ document_end	Extension sandbox. Orchestrator, vision, strategy, mouse controller, HUD. Talks to MAIN via window.postMessage bridge (internal.js).
Service worker	background/background.js	Settings storage defaults, toolbar badge ON/OFF, message handlers GET/UPDATE_SETTINGS.
Popup	popup/*	User toggles: bot enable, strategy preset, click speed, troop %, feature flags.

Table 1. Runtime components.

3.2 Sense → Decide → Act

Sense: vision (scaled canvas read) + optional internal state (balance, territory, neighbors, density).
Decide: phase policy (OPENING / LAND_RUSH / CONTESTED / LATE), free-land ratio, adaptive commit %, multi-front count, target selection (perimeter foreign cells only).
Act: (1) internal attackViaHg / ship / native expand, else (2) fireAttack two-step canvas sword on safe screen coordinates.

3.3 Data flow (simplified)

```
raf loop → AdaptiveScheduler(uncapped) → VisionEngine.processFrame
→ CoordSystem + Border/Region/Neutral → HardMode / Economy commit ratio
→ InternalActuator.attack OR MouseController.fireAttack
→ MAIN: setTroopRatio(storage182 + aS.di + hd-patch) then hg(il, jd)
→ HUD telemetry update
```

4. Repository Structure and Module Inventory

4.1 Directory tree

```
territorial-chrome-extension/
■■■ manifest.json # MV3 config, hosts, script order
■■■ README.md
■■■ background/background.js # service worker
■■■ popup/ # UI controls
■■■ icons/ # 16/48/128
■■■ docs/SOURCE_MAP.md # dump ↔ live symbol map
■■■ scratch/test_live_engine.html
■■■ content/ # all in-page logic
■■■ main-hook.js # MAIN world brain (~1.2k LOC)
■■■ content.js # orchestrator (~1.2k LOC)
■■■ vision.js # CV pipeline
■■■ controller.js # sword click actuator
■■■ internal.js # postMessage bridge
■■■ hardmode.js / strategy.js / economy.js / neutral.js
■■■ coords / border / region / grid / pathfinding / ...
■■■ hud.js + content.css
```

4.2 Module inventory (by responsibility)

File	~LOC	Responsibility
main-hook.js	1247	MAIN patch, troop force, adjacency, hg/pZ, state, suppress username overlay
content.js	1216	Master orchestrator, arm/disarm, pipeline, hotkeys, HUD feed
vision.js	647	Canvas read (no game getContext), classify, morphology, edges

File	~LOC	Responsibility
strategy.js	598	Named strategies / planners
hardmode.js	439	VH difficulty tables, commit helpers
border.js	427	Border stats / frontier cells
economy.js	362	Balance estimate, density, multi-wave config
controller.js	361	Canvas MouseEvents, fireAttack 2-step, safe chrome
region.js	354	Connected components / clusters
neutral.js	342	Free-land awareness, phase policy
pathfinding.js	337	Grid path utilities
world.js	330	Match-time world model
enemy.js	307	Opponent tracking / threat scores
coords.js	296	Screen↔grid, safe zone, UI chrome detector
hud.js	289	On-page telemetry panel
grid.js	258	Occupancy grid from vision
internal.js	211	Isolated→MAIN RPC
optimization.js	193	Memory pools, uncapped AdaptiveScheduler
heatmap.js	171	Threat field
spatial.js	156	Spatial helpers
prediction.js	151	Simple prediction
smoothing.js	137	Temporal target lock
background.js	83	Storage + badge

Table 2. Approximate line counts (source files).

4.3 Content-script load order

Isolated scripts load in manifest order so dependencies exist before the orchestrator: hardmode → coords → vision → grid → region → border → neutral → world → enemy → economy → heatmap → strategy → utility → prediction → smoothing → pathfinding → spatial → controller → internal → hud → optimization → **content.js** last.

5. Game Reverse Engineering and Source Map

A readable dump (source code.html) plus live territorial.io mapping underpins the MAIN hook. Key dump concepts:

Dump symbol	Meaning	Live mapping (desktop)
ap	Local player index	aE.et
aq[p]	Troop balance	ah.hB
b1 / empty	Uncaptured land	aE.f6 (human hg target)
cE(g,k,y,l)	Attack with tax	underlying spend path
dF	Expand empty first	prefer free land policy

Dump symbol	Meaning	Live mapping (desktop)
dJ	Enemy attack / crush	weak/closest + crush rule
dD	Bot tick order	expand → fight
dU index 5	Very Hard tables	DIFF=5 defaults
human attack	—	bB.hZ.hg(il, jd)
ship attack	sea islands	bB.hZ.pZ(il, pa) when aE.i3
neighbors	adjacency graph	ae.gG / gL / gM / jp

Table 3. Dump ↔ live symbol map (abridged). Full detail in docs/SOURCE_MAP.md.

5.1 Attack tax and crush (source spirit)

```

tax ≈ al(3 * balance, 256) // ~1.2% of balance
spent = floor(balance * (il+1) / 1024)
il = floor(ratio*1024+0.5)-1 ∈ [0,1023]
crush: if al(myBal,8) > enemyBal → troops = max(troops, al(11*enemyBal,5))

```

5.2 Very Hard behaviors copied

- Empty land first while free land is meaningful (~>2.5%).
- When fighting, high bias to weakest neighbor (~90% in dump dI[5]).
- Crush finish when much stronger than foe.
- Open commit roughly 40–45% early; later sustain lower / adaptive.
- Multi-front pressure (up to ~6 on VH tables; vision path often uses 2–3 for reliability).

6. Match Lifecycle and Arming Model

6.1 States

State	Condition	Bot behavior
STANDBY	No pending spawn, not armed	Idle loop; no vision thrash; HUD shows menu standby
PENDING SPAWN	User clicked map after Play	Light vision until mine pixels confirm near click
ARMED	Territory confirmed near spawn	Full pipeline every rAF; attacks allowed
DISARMED	No territory ~2.5s / reset	Stop attacking; avoid lobby UI clicks
USER HOLD	Trusted pointerdown	Queue cleared; bot waits

Table 4. Lifecycle states.

6.2 Why spawn-gated?

Early versions risked clicking main-menu buttons (Singleplayer / Multiplayer / name field). Arming only after a deliberate map spawn keeps the bot out of menus and prevents false attacks on lobby chrome.

7. Sensing: Vision, Grid, Borders, Coordinates

7.1 VisionEngine

- Never calls `getContext` on the game canvas (would black-screen the game). Only `drawImage(game → offscreen)` and `readPixels` from offscreen.
- Default `scaleFactor` 0.25 (4× downsample) for speed.
- Classifies pixels into unknown / water / neutral / mine / enemy histogram buckets.
- Morphological closing on player territory; Sobel-ish edge magnitude for borders.
- Reports `visionFPS` measured over 1s windows.

7.2 CoordSystem

Single source of truth for screen ↔ vision-grid mapping. Provides `isSafeScreenPoint` and `isUiChromePoint` so attack clicks never hit top chrome, bottom troop bar, or bottom-left Quit logo region.

7.3 Border / neutral / enemy

- Border detector: frontier cells and territory area estimates.
- `NeutralLandEngine`: free-land ratio, phase labels, `multiFront` suggestions.
- `EnemyTracker` + heatmap: opponent clusters and danger scores for HUD / targeting bias.

8. Decision: Strategy, Economy, Hard-Mode Profiles

8.1 Phase policy (neutral.js spirit)

Phase	Prefer	Intent
OPENING	Neutral expand	Grab free land fast after spawn
LAND_RUSH	Neutral	Keep expanding while map open
CONTESTED	Mix	Still expand if free; else fight
LATE / PRESSURE	Enemy	Crush / pressure when free land scarce
SURVIVE	Conserve	Lower commit if collapsing

Table 5. Strategic phases.

8.2 Adaptive troop commit

Commit ratio is situation-aware and EMA-smoothed, then hard-clamped roughly to **12–45%** to avoid the historical “stuck ~79% death spiral.” Manual hotkeys: C ≈ 25%, V ≈ 50% soft preference, B = full adaptive hard ratio. Popup slider 0 = full auto.

8.3 Target selection rules

- Only perimeter foreign cells (vision types neutral=2 / enemy=4) that are 4-connected to mine.
- Pack screen coordinates via `gridToScreen`; reject if outside safe map rect.
- Multi-wave: typically 2–3 fronts early for sword reliability; internal path can push more.

9. Actuation: Internal API vs Canvas Sword Attacks

9.1 Internal path (preferred)

MAIN `attackViaHg(ratio, targetId)`:

- Resolves local player; requires alive + land-adjacent target (strict).

- setTroopRatio then bB.hZ.hg(il, jd).
- Ships: bB.hZ.pZ when water maps enable aE.i3 and target is not land-adjacent.
- Native expand/crush helpers when exports exist, still adjacency-gated.

9.2 Canvas sword path (fallback / vision)

Live game land attack is a **two-click** UI sequence on #canvasA:

- Click 1 on foreign land → green radial sword button (menu aM / rb[0]).
- Click 2 near same point (~16 ms later) → confirm → troops fire.
- Events are MouseEvents tagged as bot; dispatched to canvas only (game listens on canvasA).
- force flag ignores user-hold only for confirm; UI chrome is still always blocked.

9.3 User pointer respect

Trusted pointerdown sets userPointerDown; bot clears queues and yields. This prevents fighting the human mid-micro.

10. Troop Percentage Control

The game stores troop spend code at storage index **182** (0..1023). Displayed bar ratio is approximately $(value+1)/1024$. Live attack spend uses aS.hd() derived from that value.

10.1 Force path (v9.9+)

- writeTroopStorage / bm.pW.pX(182, ilCode)
- reloadSliderFromStorage → aS.di() (and resize) so the bar redraws
- ensureHdPatch so aS.hd() returns forced il even if UI lags
- Bar-drag simulation is DISABLED (was a root cause of Quit logo hits / username popup)

10.2 Why not only write storage?

Writing data[182] alone does not always move the visible bar or the live spend code until di()/hd() paths refresh. The patch + di combination keeps attacks honest to the adaptive %.

11. UI Chrome Safety and Username-Popup Prevention

11.1 Root cause analysis

Territorial's in-game bottom-left control (module fW in the dump) is a **logo / menu button**:

- First click opens Quit Game / Surrender / Statistics.
- Second click on Quit calls jA() → jD.bi() which sets main-menu state and shows the absolute-positioned username INPUT.
- Bot perimeter attacks near the lower-left map, or synthetic bar-drag near the bottom row, could hit this control repeatedly—appearing as “username window keeps popping during game.”

11.2 Mitigations in v9.9.1+

- coords.isUiChromePoint: block top strip, bottom bar row (~y>90%), bottom-left corner (x<16% & y>78%), bottom-right chrome.

- `controller.isMapSafeClick` always enforced—even for sword confirm force clicks.
- `packPerimeterClick` rejects unsafe screen points before enqueue.
- No `window.dispatchEvent` for bot mice (canvas only).
- Bar-drag disabled; troop force via `storage+di+hd` only.
- `suppressUsernamePopup` / `suppressUsernameOverlay` removes stray `body>input[type=text]` while match armed / territory alive.

12. Performance Model (Uncapped FPS)

Earlier builds hard-capped the decision loop at ~15–30 FPS and attack pulses at ~75–85 ms, which felt sluggish. **v9.9.2** removes artificial ceilings:

Subsystem	Previous	Current
AdaptiveScheduler	15–30 FPS interval gate	<code>targetFPS=0</code> → every rAF
Attack pulseMs	70–90 ms	0 (no intentional wait)
Controller min interval	35–80 ms floor	0 default
Sword confirm delay	45 ms + long queue waits	~16 ms + short chain wait
HardMode pulse tables	85–450 ms	0 across difficulties

Table 6. Performance uncap summary.

Throughput is now limited by **display refresh** (`requestAnimationFrame`) and **CPU cost of vision + logic** on the user's hardware—not a fixed bot FPS budget. Disarm timeouts are time-based (~2.5 s without territory) so higher FPS does not prematurely end matches.

13. User Interface: HUD, Popup, Hotkeys

13.1 In-page HUD

`hud.js` injects a compact dashboard: version, FPS, state label, free-land %, territory, density, troop balance, enemy count, target coords, policy/mode, and hints (e.g., `STANDBY` → click spawn).

13.2 Extension popup

- Master bot toggle (badge ON).
- Strategy presets: Expansionist / Aggressive / Defensive.
- Click speed slider (legacy; high values map to uncapped pacing).
- Troop percentage slider (0-ish / adaptive preferred in orchestrator defaults).
- Feature toggles for `expand/attack/jitter/hotkeys` depending on popup version.

13.3 Hotkeys (page focused)

Key	Action
Z	Toggle bot enabled
C	Soft-cap commit ~25%
V	Soft-cap commit ~50%
B	Full adaptive / hard ratio

Table 7. Hotkeys.

14. Installation, Operation, and Debugging

14.1 Install / reload

- Open chrome://extensions → enable Developer mode.
- Load unpacked → select territorial-chrome-extension folder (or Reload after code changes).
- Open a fresh tab to https://territorial.io and hard-refresh (Cmd+Shift+R / Ctrl+Shift+R).
- Play → click spawn on the map (not menus) → bot arms when territory is confirmed.

14.2 Debug attributes / console

```
document.documentElement.getAttribute('data-tio-internal') // want "1" when ready
document.documentElement.getAttribute('data-tio-paths') // e.g. hg, hg+cE
document.documentElement.getAttribute('data-tio-troop') // forced %
document.documentElement.getAttribute('data-tio-troop-path')// storage182+aS.di+hd-patch
window.__TIO_HOOK_API__ && window.__TIO_HOOK_API__.state()
```

data-tio-internal	Meaning
0	Script patch missed — new tab + hard refresh
patched	Export injected; game not ready / not in match
1	Ready — internal attacks available

Table 8. Hook readiness signals.

14.3 Permissions

manifest permissions: **storage**, **activeTab**. Host permissions limited to territorial.io and localhost patterns for development.

15. Version History and Problem Log

The project evolved through many iterative fixes. Highlights relevant to current design:

Issue	Symptom	Resolution direction
Black screen on first open	Game canvas context stolen / double inject	Never getContext(game); safe MAIN patch only
Bot before spawn	Clicks menus	Arm only after Play + spawn pixel confirm
Passive bot	Few attacks	VH tables, click fallback, higher aggression
Stuck ~79% troops	Death spiral spends	storage182 + aS.di + hd-patch; clamp 12–45%
Non-adjacent attacks	No troops / invalid	Strict neighbor / 4-conn perimeter only
Sword opens only	No spend	Two-click fireAttack confirm
Username window mid-game	Quit logo double-hit → jD.bi()	Chrome exclusion; no bar-drag; strip INPUT
Low bot FPS	30 FPS + 75 ms pulse	Uncapped rAF scheduler (v9.9.2)

Table 9. Problem → fix map.

15.1 Recent versions

- v9.0.x — source-faithful MAIN brain, dump mapping.
- v9.4+ — adjacency + ships awareness.
- v9.9.0 — ASAP post-spawn fire; faster sword chain.
- v9.9.1 — username popup fix (Quit chrome + no bar-drag).
- v9.9.2 — uncapped FPS / hardware-limited loop.

16. Limitations, Risks, and Future Work

16.1 Limitations

- Live game is minified and updates can break symbol names; hooks need maintenance.
- Vision at 0.25 scale can misclassify colors on some maps / themes.
- Sword path is UI-coupled; timing still needs a small confirm delay.
- Internal path may be unavailable if patch fails (CSP, load race, game update).
- Not designed as a multiplayer cheat product; ethics and ToS are the user's responsibility.
- Popup defaults and orchestrator defaults may diverge slightly (clickSpeed legacy).

16.2 Risks

- High uncapped CPU use on weak machines (full vision every frame).
- Any residual UI chrome hit could still open menus if geometry changes.
- Removing username INPUT mid-menu could confuse if disarm logic fails.

16.3 Future work candidates

- Auto-discover live symbols when minifier renames (heuristic export finder).
- Optional lower vision scale / partial ROI capture for laptop battery mode.
- Richer ship naval strategy on water maps.
- Unit tests against recorded vision fixtures + mock `__TIO_GAME__`.
- Sync popup defaults with orchestrator uncapped settings.
- Optional shadow-DOM HUD isolation polish.

17. Appendix: Key Symbols and Quick Reference

A. Globals and DOM flags

Symbol / attribute	Meaning
<code>window.__TIO_GAME__</code>	MAIN export of live game handles
<code>window.__TIO_HOOK_API__</code>	Debug API (state, etc.)
<code>window.__TIO_FORCE_IL__</code>	Forced troop code
<code>data-tio-internal</code>	Hook readiness
<code>data-tio-troop / -path / -live</code>	Troop force telemetry
<code>__tioBotEvent</code>	Tag on synthetic mouse events

Table 10. Debug surfaces.

B. Safe-click chrome regions (approx. normalized)

- Top: $ny < 0.06$
- Bottom bar: $ny > 0.90$
- Bottom-left Quit stack: $nx < 0.16$ and $ny > 0.78$
- Bottom-right chrome: $nx > 0.90$ and $ny > 0.82$
- Playable map roughly inside 4–97% x and 7–88% y after chrome tests

C. Document control

Field	Value
Project name	Territorial.io Autonomous Human-Level Agent
Extension version	9.9.2
Manifest	3
Primary language	JavaScript (browser extension)
Report format	PDF (reportlab)
Report location	/Users/phamvotriduc/Desktop/territorial-chrome-extension/docs/Territorial_IO_Project_Report.pdf
Companion doc	docs/SOURCE_MAP.md

Table 11. Document control.

End of report. This document is generated from the codebase structure, README, SOURCE_MAP, and implemented behavior as of the version listed on the cover.